
Comprehensive Notes: Apache Airflow, ETL/ELT, Data Warehousing, OLAP, OLTP, and Data Modelling

Apache Airflow – Workflow Orchestration Tool

◆ What Is Apache Airflow?

Apache Airflow is an open-source platform developed by Airbnb and later donated to the Apache Software Foundation. It is designed to help data engineers and analysts **automate, schedule, and monitor workflows**—especially those involving data pipelines.

Airflow allows you to define workflows as **DAGs (Directed Acyclic Graphs)** using Python code. Each DAG represents a pipeline of tasks with dependencies, and Airflow ensures they run in the correct order and at the right time.

◆ Why Use Airflow?

- **Automation:** Replace manual scripts and cron jobs with automated workflows.
 - **Observability:** Track task status (success, failure, retries) via logs and dashboards.
 - **Scalability:** Run tasks on distributed systems using executors like Celery or Kubernetes.
 - **Flexibility:** Write workflows in Python, integrate with databases, cloud services, and big data tools.
 - **Extensibility:** Create custom operators and plugins to suit your needs.
-

◆ Core Components of Airflow

1. DAG (Directed Acyclic Graph)

- A DAG is a Python object that defines the structure of your workflow.
- It specifies the tasks and their dependencies.
- DAGs are acyclic, meaning tasks cannot loop back to earlier tasks.

2. Tasks

- Each task is a unit of work, such as running a script, querying a database, or sending an email.

- Tasks are defined using **operators**.

3. Operators

Operators are templates that define what a task does. Common types include:

- Python Operator: Executes a Python function.
- Bash Operator: Runs shell commands.
- Email Operator: Sends email notifications.
- Dummy Operator: Placeholder for testing or branching.
- BranchPythonOperator: Allows conditional branching in workflows.

4. Scheduler

- The scheduler monitors DAGs and triggers tasks based on their schedule.
- Uses cron expressions or presets like @daily, @hourly, @weekly.

5. Executor

- The executor determines how tasks are run:
 - SequentialExecutor: Runs one task at a time (good for testing).
 - LocalExecutor: Runs multiple tasks in parallel on a single machine.
 - CeleryExecutor: Distributes tasks across multiple workers.
 - KubernetesExecutor: Runs tasks in isolated containers.

6. Web UI

- A browser-based dashboard to monitor DAGs, view logs, trigger tasks manually, and manage configurations.

◆ Example: Daily Sales Report Pipeline

Scenario: You work for a retail company and need to automate a daily sales report.

Workflow Steps:

1. **Extract** sales data from a MySQL database.
2. **Transform** the data using Python (cleaning, aggregating).
3. **Load** the final report into Google Sheets or email it to stakeholders.

DAG Code:

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime


def extract(): pass

def transform(): pass

def load(): pass


with DAG('daily_sales_report', start_date=datetime(2023, 1, 1), schedule_interval='@daily')
as dag:

    t1 = PythonOperator(task_id='extract', python_callable=extract)

    t2 = PythonOperator(task_id='transform', python_callable=transform)

    t3 = PythonOperator(task_id='load', python_callable=load)


t1 >> t2 >> t3
```

ETL vs ELT – Data Integration Workflows

◆ ETL: Extract → Transform → Load

Definition:

ETL is a traditional data integration process where data is extracted from source systems, transformed externally (often using Python, Spark, or ETL tools), and then loaded into a data warehouse.

Detailed Steps:

1. Extract:

- Pull data from sources like relational databases, APIs, flat files, or cloud storage.
- Handle different formats (CSV, JSON, XML).

2. Transform:

- Clean data (remove duplicates, handle missing values).

- Apply business logic (e.g., currency conversion, date formatting).
- Aggregate or join datasets.
- Validate and enrich data.

3. Load:

- Push the transformed data into a target system like PostgreSQL, Oracle, or a data warehouse.

Use Cases:

- When transformation must happen before loading due to schema constraints.
- Legacy systems with limited compute power.

Tools:

- Talend
- Informatica
- Apache NiFi
- AWS Glue
- Apache Airflow (as orchestrator)

◆ **ELT: Extract → Load → Transform**

Definition:

ELT is a modern approach where raw data is loaded directly into the warehouse and transformed using the warehouse's compute power.

Detailed Steps:

1. Extract:

- Pull raw data from sources (same as ETL).

2. Load:

- Load raw data directly into the data warehouse (e.g., Snowflake, BigQuery).

3. Transform:

- Use SQL or tools like dbt to clean, join, and aggregate data inside the warehouse.

Use Cases:

- Cloud-native platforms with scalable compute.
- When storing raw data is beneficial for auditing or reprocessing.

Tools:

- dbt (Data Build Tool)
- SQL scripts
- Airflow (as orchestrator)

◆ ETL vs ELT Comparison

Feature	ETL	ELT
Transformation	Before loading	After loading
Speed	Slower for large data	Faster with cloud scalability
Flexibility	Less flexible	More flexible
Tools	Talend, Informatica, Airflow dbt, Snowflake, BigQuery	
Use Case	Legacy systems	Modern cloud platforms

Data Warehousing

◆ Definition

A data warehouse is a centralized system that stores integrated, historical data from multiple sources. It is optimized for analytical queries, reporting, and decision-making.

◆ Characteristics

- **Subject-oriented:** Organized by business domains (e.g., sales, finance).
- **Integrated:** Combines data from various sources with consistent formats.
- **Time-variant:** Stores historical snapshots for trend analysis.
- **Non-volatile:** Data remains stable once entered; no frequent updates.

◆ Architecture Components

1. **Data Sources:** ERP, CRM, flat files, APIs.
2. **ETL Process:** Extract → Transform → Load.

3. **Staging Area:** Temporary raw data storage.
 4. **Data Warehouse:** Central repository for cleaned and structured data.
 5. **Data Marts:** Department-specific subsets of the warehouse.
 6. **BI Tools:** Reporting and visualization (e.g., Tableau, Power BI).
-

OLTP (Online Transaction Processing)

◆ **Purpose**

Handles real-time transactional data such as banking, retail, and booking systems.

◆ **Features**

- High volume of short transactions.
- Fast query processing.
- Normalized schema to reduce redundancy.
- Frequent updates and inserts.

◆ **Examples**

- ATM withdrawals.
 - Online orders.
 - Flight bookings.
 - Inventory updates.
-

OLAP (Online Analytical Processing)

◆ **Purpose**

Supports complex queries and analysis on large volumes of historical data.

◆ **Features**

- Low volume of complex queries.
- Read-heavy operations.
- Denormalized schema for faster querying.
- Multidimensional modeling.

◆ OLAP Operations

- **Slice:** Filter data on one dimension (e.g., sales in January).
- **Dice:** Filter on multiple dimensions (e.g., sales in January for Region A).
- **Drill Down:** View data at a finer level (e.g., from year → month → day).
- **Roll Up:** Aggregate data to a higher level (e.g., day → month → year).
- **Pivot:** Rotate data axes for different perspectives.

◆ Example

Analyzing monthly sales by region and product category using a dashboard.

OLTP vs OLAP Comparison

Feature	OLTP	OLAP
Purpose	Transactional processing	Analytical processing
Data Volume	High (real-time)	Large (historical)
Schema	Normalized	Denormalized
Operations	Insert, Update, Delete	Select, Aggregate
Users	Clerks, DB admins	Analysts, decision-makers
Speed	Fast for small queries	Optimized for complex queries

Data Modeling (continued)

◆ Types of Data Models

1. Conceptual Data Model

- High-level representation of business entities and their relationships.
- Focuses on what data is important, not how it's stored.
- Used during initial planning and stakeholder discussions.
- Example: Entities like Customer, Product, Order with relationships like "Customer places Order".

2. Logical Data Model

- More detailed than conceptual.
- Specifies attributes, primary keys, foreign keys, and relationships.
- Independent of physical implementation.
- Example: Customer table with attributes like CustomerID, Name, Email.

3. Physical Data Model

- Actual implementation in a database system.
 - Includes table structures, data types, indexes, constraints.
 - Tailored to specific DBMS (e.g., MySQL, PostgreSQL).
 - Example: CustomerID as INT, Name as VARCHAR(100), Email as VARCHAR(255).
-

Schema Types in Data Warehousing

Star Schema

- Central **fact table** surrounded by **dimension tables**.
- Dimension tables are denormalized for fast querying.
- Easy to understand and implement.
- Example: Sales fact table linked to Product, Customer, Time, Region dimensions.

Snowflake Schema

- Extension of star schema where dimension tables are normalized.
- Reduces data redundancy but increases query complexity.
- Example: Product dimension split into Category and Supplier tables.

Galaxy Schema (Fact Constellation)

- Multiple fact tables share dimension tables.
 - Used in complex systems with multiple business processes.
 - Example: Sales and Inventory fact tables sharing Product and Time dimensions.
-

Fact and Dimension Tables

Fact Table

- Contains **quantitative data** for analysis (e.g., sales amount, quantity sold).
- Includes foreign keys to dimension tables.
- Usually very large in size.

◆ Dimension Table

- Contains **descriptive attributes** (e.g., product name, region, customer type).
- Helps filter, group, and label facts.
- Typically smaller and used for slicing and dicing data.



Types of Facts

- **Additive Facts:** Can be summed across all dimensions (e.g., revenue, units sold).
- **Semi-additive Facts:** Can be summed across some dimensions (e.g., account balance over time but not across accounts).
- **Non-additive Facts:** Cannot be summed (e.g., percentages, ratios).



Normalization vs Denormalization

◆ Normalization

- Organizes data to reduce redundancy.
- Data is split into multiple related tables.
- Used in OLTP systems.
- Improves data integrity and consistency.

◆ Denormalization

- Combines tables to reduce joins and improve query performance.
- Used in OLAP systems.
- Increases redundancy but speeds up analytical queries.

Feature	Normalization	Denormalization
Purpose	Reduce redundancy	Improve query performance
Used In	OLTP systems	OLAP systems

Feature	Normalization	Denormalization
Structure	Many small tables	Fewer, larger tables
Query Speed	Slower for complex queries	Faster for analytical queries

Slowly Changing Dimensions (SCD)

Used to manage changes in dimension data over time. For example, if a customer changes their address or marital status.

◆ Types of SCD

- **Type 1:** Overwrites old data with new data.
 - No history maintained.
 - Simple but loses historical context.
 - **Type 2:** Adds a new row with versioning or effective dates.
 - Preserves full history.
 - Common in analytical systems.
 - **Type 3:** Adds a new column to store previous value.
 - Limited history (only one previous state).
 - Useful when only recent changes matter.
-

ETL Workflow in Data Warehousing

◆ Step-by-Step Breakdown

1. Extract:

- Connect to source systems (e.g., MySQL, APIs, flat files).
- Pull raw data in various formats (CSV, JSON, XML).
- Handle authentication, pagination, and error handling.

2. Transform:

- Clean data: remove nulls, duplicates, outliers.
- Apply business rules: currency conversion, date formatting.

- Enrich data: add calculated fields, join with reference tables.
- Validate: ensure schema consistency and data quality.

3. Load:

- Insert transformed data into staging or final warehouse tables.
- Use bulk loading for efficiency.
- Monitor for failures and retries.

◆ Tools for ETL

- **Apache Airflow:** Orchestration and scheduling.
- **Talend / Informatica:** GUI-based ETL development.
- **AWS Glue:** Serverless ETL in the cloud.
- **Python / Pandas / SQL:** Custom transformations.